AutoComplete Search Engine

Full-Stack System — C++ Backend + Web Frontend

Data Structures & Algorithms

TRIE DATA STRUCTURE | C++17 | REST API

Team Members:

Shivom Singh
Aditya Prakash
Shelar Yash Kanifnath
Tumma Abhinav

The Problem with Traditional Search

Modern applications demand fast, intelligent search — but conventional methods fall short under real-world conditions.

No Real-Time Response

Results only appear after a full query is submitted — no per-keystroke feedback.

No Typo Tolerance

Exact-match logic fails silently when users make even minor spelling errors.

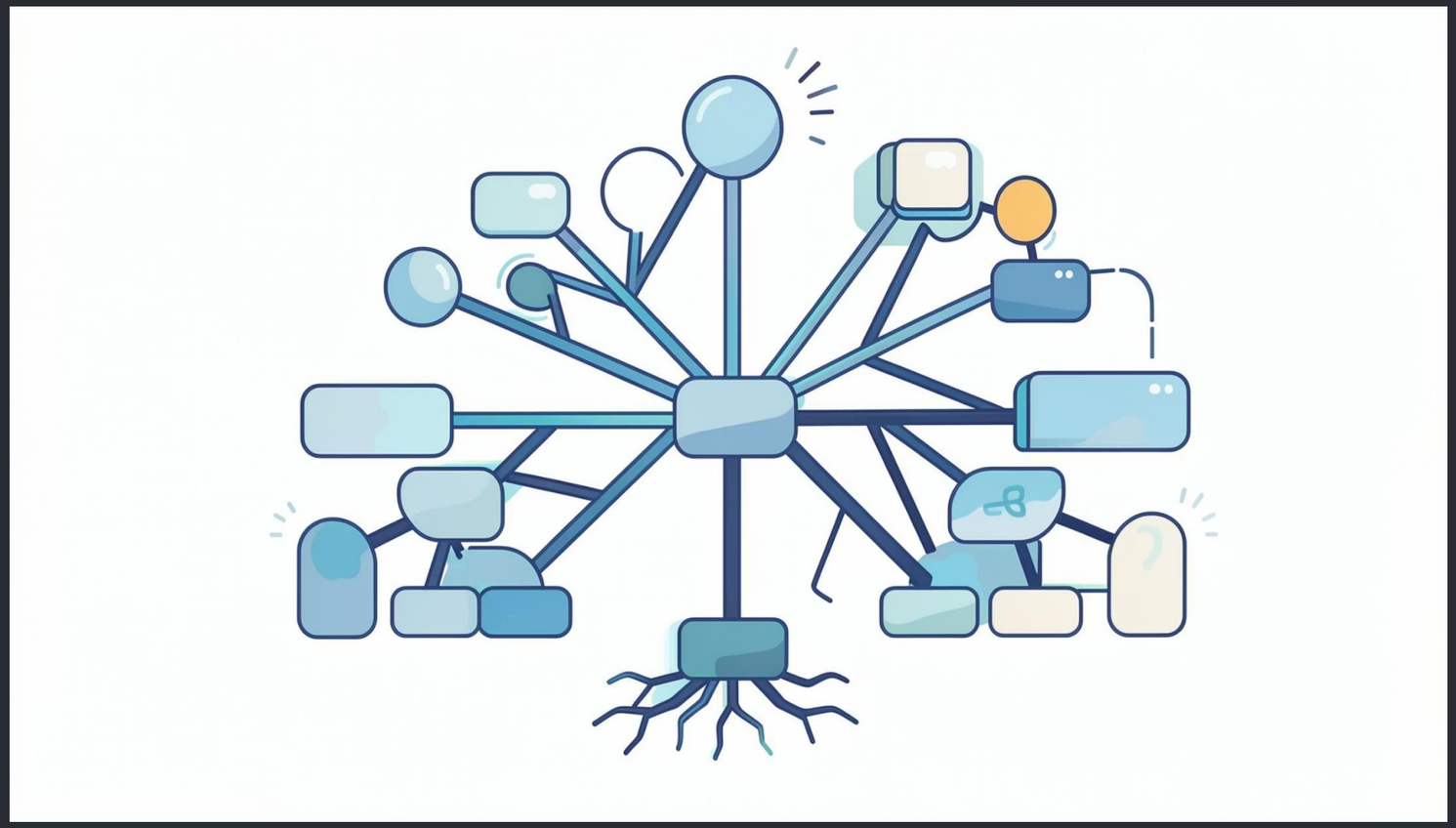
Slow Prefix Queries

Linear scans through large datasets are far too slow for interactive use.

No Dynamic Updates

Static indexes can't incorporate newly inserted words at runtime.

What is a Trie?



A **Trie (Prefix Tree)** is a tree-based data structure purpose-built for string storage and retrieval.

→ Character Nodes

Each node represents a single character along a path.

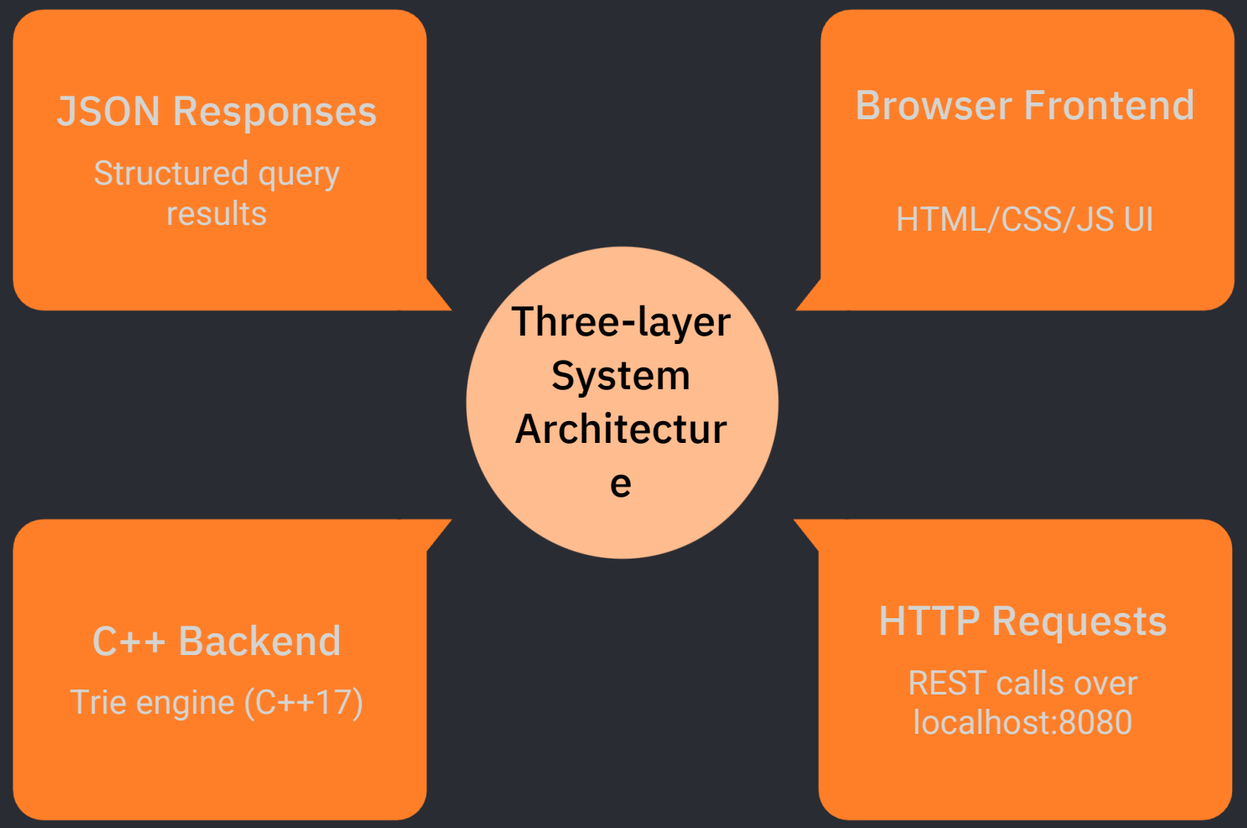
→ Shared Prefixes

Common prefixes are stored once, reducing memory redundancy.

→ O(L) Lookup

Search time depends only on word length *L*, not dataset size.

System Architecture & Tech Stack



The client-server model keeps the Trie engine fully decoupled from the UI layer, enabling clean REST communication over localhost:8080.

Data Structure

Trie — C++ with pointer-based nodes

Backend

C++17 + cpp-httplib HTTP server

Frontend

HTML, CSS, Vanilla JavaScript

API

REST endpoints returning JSON

Core Features

Real-Time Autocomplete

Suggestions update on every keystroke using `_getch()` — no Enter required.

Frequency Tracking

Each word stores a usage counter. Searches increment frequency dynamically.

Fuzzy Spell Correction

Levenshtein distance (≤ 2 edits) surfaces the most relevant corrections for typos.

Dynamic Word Insertion

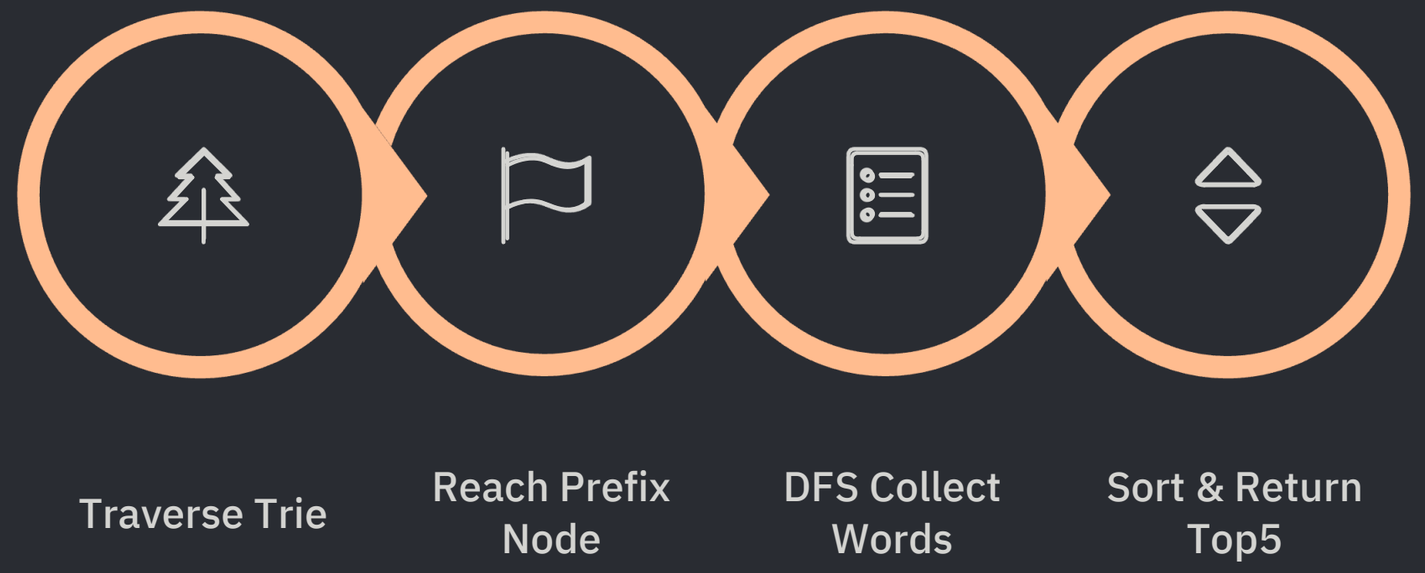
New words can be added at runtime and immediately become searchable.

Frequency-Based Ranking

Top-K suggestions are sorted by frequency so the most relevant results appear first.

Key Algorithms

Autocomplete — O(L)



Fuzzy Correction — Levenshtein DP

For a mistyped query, the engine compares it against every stored word using a dynamic programming matrix:

```
dp[i][j] = 1 + min(
  dp[i-1][j-1],  // replace
  dp[i-1][j],    // delete
  dp[i][j-1]     // insert
)
```

Only candidates with **edit distance ≤ 2** are surfaced, capped at the top 5 by frequency.

Trie Node & Insert Operation

Node Structure

```
class TrieNode {
public:
  TrieNode* children[26];
  bool isEnd;
  int frequency;
};
```

`children[26]` — one slot per lowercase letter
`isEnd` — marks a complete word boundary
`frequency` — tracks search usage count

Insert — O(L)

```
if (current->children[index] == NULL) {
  current->children[index] = new TrieNode();
}
current = current->children[index];
```

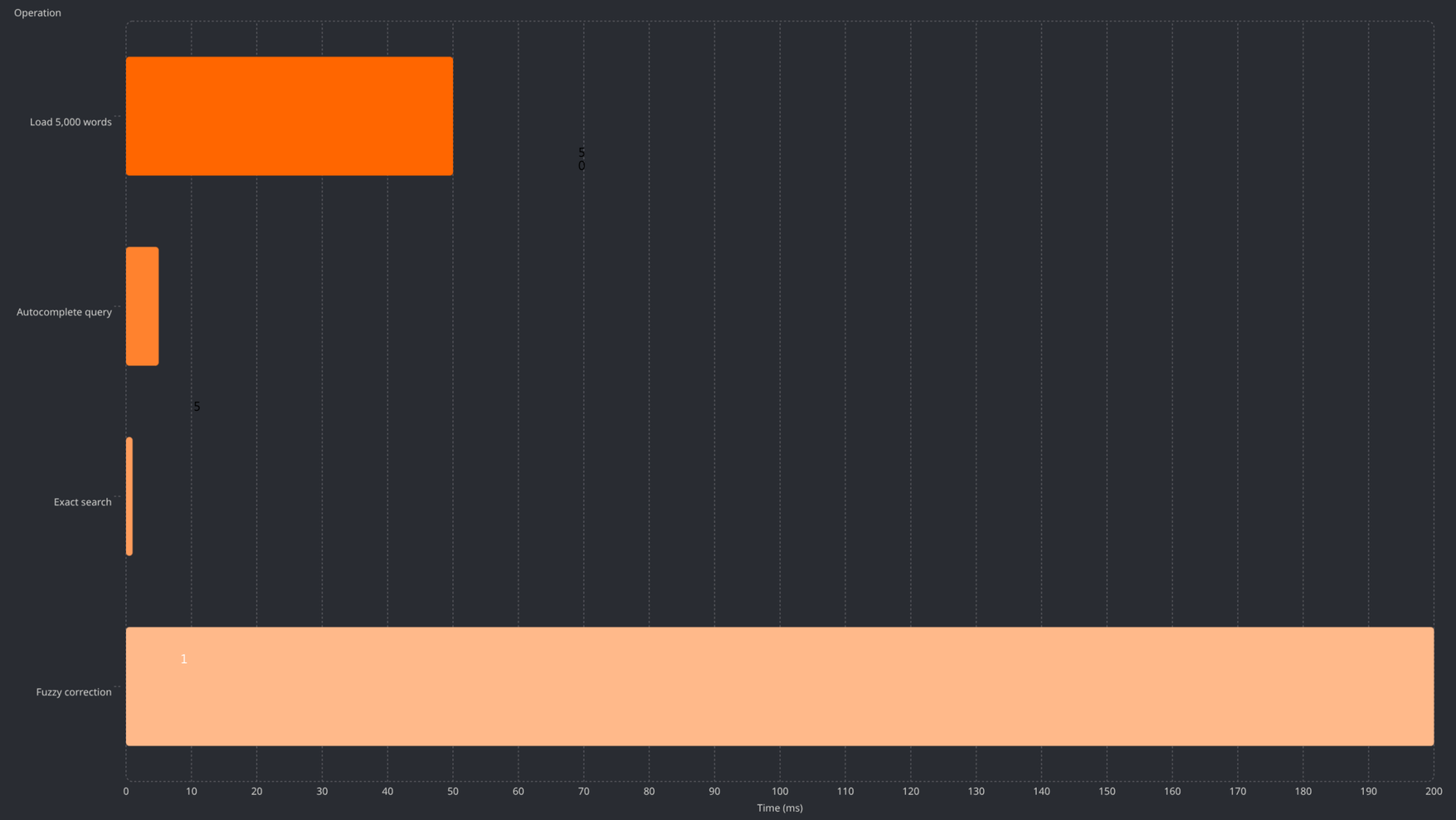
Characters are traversed one at a time. New nodes are allocated only when a path doesn't already exist, naturally enabling prefix sharing across words.

API Endpoints

All endpoints are served over HTTP and respond with **JSON-formatted** payloads, making the backend consumable by any frontend or tool.

Endpoint	Function	Description
`/suggest`	Autocomplete	Returns top-K prefix matches ranked by frequency
`/search`	Exact Lookup	Checks if a word exists; increments its frequency
`/correct`	Typo Correction	Returns fuzzy matches within edit distance ≤ 2
`/insert`	Dynamic Insert	Adds a new word to the Trie at runtime

Performance Results



<5ms

Autocomplete

Per-keystroke suggestion latency

O(L)

Search Complexity

Scales with word length, not dataset size

5K

Words Preloaded

Loaded from file in under 50 ms at startup

Conclusions & Future Scope

What We Built

- A real-world DSA application with measurable, high-performance results
- Full-stack integration from a C++ Trie engine to a live REST API
- Fuzzy correction and frequency-aware ranking beyond basic autocomplete

What's Next

- Persistent storage to retain frequency data across sessions
- Multi-language support with Unicode-aware Trie nodes
- Cloud deployment with HTTPS and authentication on public APIs
- Improved ranking via ML-based personalization models